

Trabajo Práctico N°2

Generador PRBS, Filtro IIR, Unidad de Diagnóstico (DU),
Medidor de Clock y Mapa de Registros

Curso de Diseño Digital VLSI — Fundación Fulgor

Modalidad: Grupal (mismos grupos de TP1)
Lenguaje: SystemVerilog sintetizable (estilo RTL, sin `initial`, sin `delay`)
Entregable: RTL + testbench + informe descriptivo + waveforms

Introducción

Este esquema –generar estímulo, procesarlo, y poder *mirarlo* con una ventana de captura con trigger– es exactamente el mismo patrón que se usa en un BiST de un ASIC real o en el datapath de instrumentación de un transceiver de alta velocidad: no es un ejercicio artificial, es la estructura que se van a encontrar el día de mañana en un chip.

Importante: El bloque FIR (ya lo diseñaron) lo integran como una caja negra con una interfaz dada, y se encargan de generarle el estímulo (PRBS) y de instanciar los registros de configuración de sus coeficientes.

Objetivos de aprendizaje

- Diseñar una sola PRBS, entendiendo la relación entre polinomio primitivo y realimentación en hardware.
- Diseñar una memoria de captura circular con lógica de pre/post-trigger (patrón muy usado en instrumentación y debug de ASIC).
- Diseñar un medidor de frecuencia por conteo de bordes con ventana de gate, e identificar dónde aparece un cruce de dominio de reloj (CDC) y por qué hay que tratarlo con cuidado.
- Integrar todo detrás de un mapa de registros (regmap) simple, con separación clara entre lógica de control/configuración y datapath.

1 Arquitectura general

Cada bloque es una entidad separada, con su propia sección de especificación abajo. El regmap es el único punto de entrada/salida hacia el “mundo exterior” del sistema: todo parámetro de configuración y toda señal de status pasa por ahí.

2 Bloque A - Generador PRBS parametrizable

2.1 Repaso conceptual

Un PRBS se genera con un LFSR (*Linear Feedback Shift Register*) tipo Fibonacci: un registro de N bits que en cada ciclo se desplaza, y el bit que entra por el extremo se calcula como XOR de un subconjunto fijo de bits del registro (las “tomas” o *taps*), definidas por un polinomio primitivo de grado N sobre $GF(2)$. Con tomas primitivas, la secuencia recorre las $2^N - 1$ combinaciones no nulas antes de repetirse (secuencia de máxima longitud).

2.2 Polinomios a implementar

El generador tiene que tener dos prbs que sean seleccionables, seleccionable en tiempo de configuración.

Patrón	Polinomio primitivo	Tomas (taps)
PRBS10	$x^{10} + x^7 + 1$	10, 7
PRBS11	$x^{11} + x^9 + 1$	11, 9
PRBS12	$x^{12} + x^6 + x^4 + x + 1$	12, 6, 4, 1
PRBS13	$x^{13} + x^4 + x^3 + x + 1$	13, 4, 3, 1
PRBS14	$x^{14} + x^5 + x^3 + x + 1$	14, 5, 3, 1
PRBS15	$x^{15} + x^{14} + 1$	15, 14

Tarea A

1. Implementar el generador según la especificación.
2. Testbench: para cada uno de los 2 órdenes, verificar por simulación que el período de la secuencia es efectivamente $2^N - 1$ (contar ciclos hasta que el estado vuelve a la semilla inicial). Para $N = 14, 15$ alcanza con verificar los primeros 2^{12} bits contra un modelo de referencia en Python/C (no hace falta correr el período completo).
3. Documentar en el informe cómo mapearon cada polinomio de la tabla a las tomas del `case`.

3 Bloque B - Filtro FIR (dado)

Este bloque **ya fue entregado por otro grupo**. A los fines de este TP, ustedes lo tratan como caja negra con la siguiente interfaz asumida (si el archivo real que reciban difiere, documenten la diferencia en el informe y ajusten la instancia — no modifiquen la lógica interna del filtro): Pero si en vez de 19 coeficientes, que sean mas, (tenemos 14nm ahora)

4 Bloque C - Unidad de Diagnóstico (DU): captura con pre/post trigger

4.1 Concepto

La DU se comporta como un osciloscopio digital interno: mantiene una ventana de captura de W muestras de `sample_out` (salida del filtro), organizada en dos mitades **de igual tamaño** alrededor del evento de disparo (*trigger*):

Es decir: cuando ocurre el trigger, la muestra que lo disparó queda ubicada en el centro exacto de la ventana capturada; a su izquierda quedan las $W/2$ muestras inmediatamente anteriores (que ya tienen que estar guardadas *antes* de que el trigger ocurra, porque el pasado no se puede reconstruir), y a su derecha las $W/2$ muestras siguientes (que todavía no existen al momento del trigger, y hay que esperar a que lleguen).

4.2 Especificación

- Parámetro de diseño W (potencia de 2, ej. 256), con `PRETRIGGER_LEN = POSTTRIGGER_LEN =  $W/2$` . **Ambas mitades deben ser configurables en tiempo de diseño con el mismo parámetro** para garantizar que sean simétricas por construcción (no dos parámetros independientes que el usuario podría desalinear).

- Memoria interna: buffer circular de profundidad W que se va llenando en forma continua con `sample_out` cada vez que `sample_valid` está en alto (aunque no haya ningún trigger armado: la mitad de pretrigger tiene que estar siempre disponible).
- Señal `arm` (desde el regmap): habilita la detección de trigger. Mientras `arm=0` el buffer circular sigue rotando pero no se congela nunca.
- Condición de trigger: `trigger_in` en alto (comparador externo simple, ej. `sample_out >= threshold`, con `threshold` programable desde el regmap) mientras `arm=1`.
- Al detectar trigger: la posición actual de escritura queda fijada como el centro de la ventana. El bloque sigue escribiendo `POSTTRIGGER_LEN` muestras más y después se congela (deja de aceptar nuevas muestras) y levanta `capture_done`.
- Lectura: mientras `capture_done=1`, el regmap puede leer la ventana completa muestra por muestra (dirección = índice dentro de la ventana, dato = valor de la muestra), en orden cronológico (primera dirección = muestra más vieja del pretrigger).
- `rearm` (pulso desde el regmap) vuelve a poner `capture_done=0` y habilita una nueva captura.

Tarea C

1. Diseñar el buffer circular y la máquina de estados de control (armado → esperando trigger → llenando posttrigger → capturado → rearmado).
2. Dibujar el diagrama de estados en el informe (a mano o con herramienta, no hace falta LaTeX).
3. Testbench: generar una señal de entrada conocida (puede ser directamente la salida del PRBS sin pasar por el filtro, para simplificar la verificación), forzar un trigger en un instante conocido, y verificar por simulación que la muestra del trigger efectivamente cae en el índice $W/2$ de la ventana leída.
4. Caso de borde a verificar: ¿qué pasa si llega un segundo trigger mientras se está llenando el posttrigger de una captura anterior? Definan el comportamiento (ignorarlo es una respuesta válida, pero tiene que estar documentada y verificada).

5 Bloque D - Medidor de frecuencia de clock

5.1 Concepto

Se pide medir la frecuencia de un reloj de dominio desconocido (`clk_in`, puede ser asíncrono respecto del reloj de referencia `clk_ref`) contando flancos durante una ventana de tiempo de referencia (*gate*) generada a partir de `clk_ref`: si se cuentan M flancos de `clk_in` durante una ventana de T_{gate} segundos, $f_{in} \approx M/T_{gate}$.

5.2 Interfaz

```

1 module clk_meter #(
2     parameter int WIN_W = ..., // a definir por el grupo
3     parameter int CNT_W = ...  // a definir por el grupo
4 ) (
5     input  logic          clk_ref,
6     input  logic          clk_in,

```

```

7  input  logic          rst_n,
8  input  logic [WIN_W-1:0] window_len,
9  output logic [CNT_W-1:0] count,
10 output logic          done
11 );

```

`window_len` (dominio `clk_ref`) programa la duración del gate en ciclos de `clk_ref`. Al cerrarse el gate, `count` (dominio `clk_ref`) queda fijo con el resultado de la última medición y `done` se levanta por un ciclo. `rst_n` es el reset del bloque, asumido asíncrono y sincronizado internamente a cada dominio que lo necesite. Los anchos `WIN_W` y `CNT_W` quedan a criterio de cada grupo: tienen que dimensionarlos para cumplir la especificación de resolución más abajo (y documentar en el informe cómo llegaron a esos valores).

5.3 Especificación

- Contador de ventana (`gate_counter`) en el dominio `clk_ref`, cargado con `window_len`.
- Contador de flancos de `clk_in`, corriendo en el dominio `clk_in`.
- **Esto es un cruce de dominios de reloj.** La señal de “gate abierto/cerrado” generada en `clk_ref` tiene que cruzar a `clk_in` para habilitar/deshabilitar el conteo, y el valor final del contador (en `clk_in`) tiene que cruzar de vuelta a `clk_ref` para quedar disponible en `count`. No conecten las dos lógicas directamente: usen el patrón de sincronización que corresponda (más abajo).
- Al cerrarse el gate, el valor contado debe quedar disponible en `count` y levantar el flag `done`.
- **Requisito de resolución:** `clk_ref` = 200 MHz, y el reloj a medir es del orden de 750 MHz. El diseño tiene que alcanzar una resolución de medición de 1 PPM (partes por millón) sobre 750 MHz. Dimensionen `window_len` y, por lo tanto, `WIN_W` en función de este requisito, y `CNT_W` en función del máximo conteo esperable durante ese `window_len`.

Tarea D

1. Identificar y dibujar en el informe **las dos travesías de CDC** de este bloque (control `ref`→`in` y dato `in`→`ref`), indicando qué tipo de señal es cada una (pulso vs. nivel vs. bus multibit) y por qué eligieron el esquema de sincronización que usaron para cada una (doble flip-flop, handshake de 4 fases, gray code, etc. — justifiquen la elección, no alcanza con nombrarla).
2. Implementar el bloque con la sincronización correspondiente.
3. Testbench con `clk_in` generado a una frecuencia conocida y *no* relacionada armónicamente con `clk_ref` (para forzar un cruce de dominios genuinamente asíncrono), y verificar que la medición cae dentro de un margen de error esperable dado el `window_len` programado.
4. **Pregunta:** ¿qué pasa si el reloj que quieren medir es de 8 GHz en lugar de 750 MHz? Analicen el problema en el informe y **impleméntenlo**.

6 Bloque E - Mapa de registros (regmap)

6.1 Interfaz de bus

Interfaz simple, síncrona, de un ciclo de latencia:

```

1 module regmap (
2     input  logic      clk,
3     input  logic      rst_n,
4     input  logic [7:0] addr,
5     input  logic [31:0] wdata,
6     input  logic      req,
7     input  logic      is_write,
8     output logic [31:0] rdata,
9     output logic      ack
10    // + puertos de configuracion/status hacia cada bloque
11 );

```

Un **req** con **is_write=1** escribe **wdata** en la dirección **addr** en el flanco siguiente; un **req** con **is_write=0** es una lectura, y entrega el dato en **rdata** con **ack** en alto un ciclo después. Un solo **req** por ciclo: no hace falta soportar lectura y escritura simultáneas (**is_write** sólo importa cuando **req=1**).

6.2 Mapa de direcciones mínimo

Addr	Registro	SW	HW	Descripción
0x00	PRBS_CTRL	R/W	R	bit0 = enable, bits[3:1] = order_sel
0x01	PRBS_SEED	R/W	R	semilla de 15 bits
0x04–0x07	IIR_COEF_A0..A3	R/W	R	coeficientes $a_0..a_3$
0x08–0x0B	IIR_COEF_B0..B3	R/W	R	coeficientes $b_0..b_3$
0x10	DU_CTRL	R/W	R	bit0 = arm, bit1 = rearm (pulso)
0x11	DU_THRESHOLD	R/W	R	umbral de trigger
0x12	DU_STATUS	RO	W	bit0 = capture_done
0x13	DU_RDADDR	R/W	R	índice dentro de la ventana a leer
0x14	DU_RDDATA	RO	W	muestra en DU_RDADDR
0x20	CLKMEAS_WINDOW	R/W	R	duración del gate (window_len), en ciclos de clk_ref
0x21	CLKMEAS_COUNT	RO	W	resultado de la última medición (count)
0x22	CLKMEAS_STATUS	RO	W	bit0 = done
0x30–0x3F	CTRL_FLAGS[0..15]	R/W	R	16 bits de propósito general, a disposición de cada grupo

Las columnas **SW** y **HW** indican el acceso desde cada lado: **SW** es lo que el regmap expone por el bus (lo que ustedes leen/escriben con **req/is_write**); **HW** es lo que hace el bloque correspondiente con ese registro puertas adentro (R = el bloque lo lee como configuración, W = el bloque lo escribe como status/dato). Del lado **SW** un registro es siempre R/W (configuración) o RO (status/dato que sólo se lee); del lado **HW** es siempre R o W, nunca ambos — un bloque no necesita leer y escribir el mismo registro, así que si SW puede escribirlo es porque HW sólo lo lee, y si SW sólo lo lee es porque HW lo escribe.

Los **CTRL_FLAGS** son deliberadamente genéricos: cada grupo puede usarlos para lo que necesite dentro de su propia integración (por ejemplo, un bit de **soft_reset** local, un modo de test adicional, etc.), pero el uso que le den tiene que quedar documentado en el informe.

Tarea E

1. Antes de escribir RTL: dibujar a mano o con wavepaint (u otra herramienta de forma de onda) una transacción de escritura y una de lectura del regmap, ciclo a ciclo (`clk`, `addr`, `wdata`, `req`, `is_write`, `rdata`, `ack`), para tener clara la relación temporal antes de diseñar la máquina de estados. Incluir el dibujo en el informe.
2. Implementar el regmap con decodificación de `addr` plana (`case` sobre dirección, sin funciones), y generar desde ahí todas las señales de configuración de los bloques A–D.
3. Ampliar el mapa si su implementación de algún bloque necesita registros adicionales (ej. si eligieron $W \neq 256$ para la DU y quieren exponer W como registro de solo lectura). Toda ampliación debe estar documentada en el informe con la misma tabla de formato.
4. Testbench de integración: desde el regmap, programar semilla y orden del PRBS, coeficientes del filtro, armar la DU con un umbral alcanzable, esperar la captura, y leer la ventana completa por el bus, verificando que el índice de trigger coincide con lo esperado.

7 Entregables

- RTL sintetizable de los 5 bloques (A, C, D, E, más la instancia del IIR dado) integrados en un `top` único.
- Testbenches de cada bloque (según lo pedido en cada Tarea) más el testbench de integración de la Tarea E.
- Memoria descriptiva breve (no más de 6–8 páginas) que incluya: mapeo de polinomios PRBS, diagrama de estados de la DU, diagrama de las travesías de CDC del medidor de clock con su justificación, y el mapa de registros final (con las ampliaciones si las hubo).
- Capturas de waveform de cada testbench mostrando el caso relevante (período del PRBS, trigger centrado en la ventana, medición de frecuencia, lectura por regmap).

Consulta: dudas de interfaz sobre el filtro IIR entregado, o cualquier ambigüedad del regmap, se resuelven en la clase de consulta — documenten en el informe cualquier decisión de diseño que hayan tomado frente a una ambigüedad, aunque después la hayamos aclarado en clase.